Machine Translated by Google

Autonomous University of Ciudad Juárez

Institute of Engineering and Technology

Department of Electrical and Computer Engineering

Task 18 IoT Irrigation System

Sensors and Instrumentation

Angel Alvidrez 225581:

Date: May 21, 2025

Machine Translated by Google

Index

1. Introduction 2

2. Theoretical Framework 3

2.1. ESP32 Microcontroller. 3

2.2. Soil Moisture Sensor 3

2.3. DHT11 Sensor 4

2.4. PIR Sensor (Motion Sensor) 4

2.5. 5V Relay Module. 5

2.6. Water Pump and Flow Control. 5

2.7. Blynk Platform. 6

2.8. LCD Screen 6

2.9. I2C Communication and PCF8574 Module. 7

3. Development 8

3.1. Microcontroller and communication. 8

3.2. Sensors and actuators 8

3.3. Program logic 8

3.4. Data Visualization 9

3.5. System operation 9

4. Code 10

5. Results 16

6. Conclusion 16

Machine Translated by Google

1. Introduction

Water scarcity and inefficient use of water resources are persistent problems.
in urban agriculture and gardening. In this context, automated irrigation systems
They represent an efficient solution for reducing water waste and improving care
of the plants.

The present project aims to develop an intelligent irrigation system using
an ESP32 development board, soil moisture sensors, and the Blynk platform for
Remote monitoring. This system allows real-time detection of humidity levels in the
ground and activate irrigation automatically when needed. In addition, the user can
View the plant's status from your mobile device, thanks to Wi-Fi connectivity
and the user-friendly interface offered by Blynk [1].

The idea was inspired by previous open-source projects, such as the one developed by
Viral Science [2], which showed a simple but functional approach to monitoring
plants through the integration of sensors and controllers connected to Blynk Starting
From this base, a customized solution was developed, adapted to the available resources.
and specific needs of the environment where it will be implemented.

Machine Translated by Google

2. Theoretical Framework

The development of intelligent systems for monitoring and controlling plants has taken of great relevance in the field of automation and the Internet of Things (IoT). These systems allow for the optimization of resource use, such as water, improving efficiency and facilitating plant care through the use of interconnected sensors and actuators.

2.1. ESP32 Microcontroller

The ESP32 is a dual-core microcontroller developed by Espressif Systems, which It integrates Wi-Fi and Bluetooth connectivity, and offers high performance with low power consumption. energetic [3]. Thanks to its versatility and processing capacity, it is widely Used in IoT projects that require wireless communication and real-time control real, like the smart irrigation system presented in this project.



Figure 1: ESP32 Development Board

2.2. Soil Moisture Sensor

The soil moisture sensor used measures the electrical conductivity between two electrodes immersed in the substrate. The resistance varies depending on the content of water, allowing the soil moisture level to be determined by means of an analog signal that the microcontroller can interpret. This data is essential to activate the water pump and maintain adequate irrigation.

Machine Translated by Google

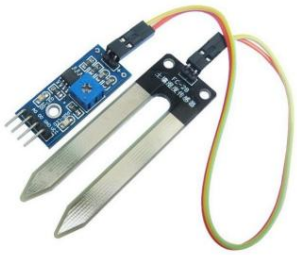


Figure 2: HR0040 Soil Moisture Sensor

2.3. DHT11 Sensor

The DHT11 sensor is a digital device that measures temperature and relative humidity.
of the air. It has a capacitive humidity sensor and a thermistor for temperature,
delivering values in digital format with sufficient resolution for applications
basic environmental conditions. In the system, this sensor allows monitoring of atmospheric conditions.
that influence the state of the plant.

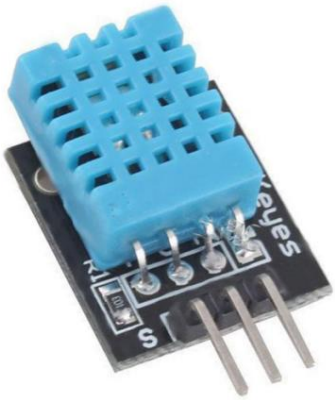


Figure 3: Air humidity and temperature sensor

2.4. PIR Sensor (Motion Sensor)

The PIR (Passive Infrared) sensor detects movement based on changes in the
Infrared radiation emitted by objects in its field of view. It is used to detect
the presence or passage of people or animals near the plant, enabling alerts and
reports in the mobile application when the sensor is active.

Machine Translated by Google



Figure 4: PIR motion sensor

2.5. 5V Relay Module

The 5V relay module allows you to control AC or DC devices
continuous higher voltage and current, such as the water pump, using the signal of
Low voltage generated by the microcontroller. This module acts as a switch
electronic shielding that isolates the microcontroller from high electrical loads, ensuring the
system security.



Figure 5: 5V relay module

2.6. Water Pump and Flow Control

The water pump used is a 5V DC pump, designed for
supply a constant flow of water suitable for watering small plants and
pots. The key feature of this pump is its nominal flow rate of approximately 1.6
liters per minute [4] (0.0267 liters per second), which allows for precise calculation of the
Total water consumption during operation. The pump is activated by

Machine Translated by Google

a 5V relay module that functions as an electronic switch, controlled directly by the ESP32 to turn the irrigation on or off according to the conditions detected by the sensors.

Precise control of water flow is essential to avoid both insufficient and excessive irrigation. excess, optimizing the use of water resources and promoting healthy development of the plants. In addition, the system calculates and displays the accumulated consumption in real time. of water during pump operation, facilitating efficient monitoring.

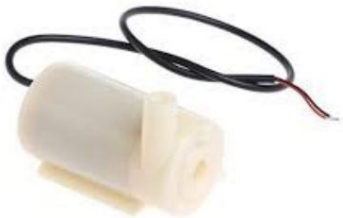


Figure 6: 9V Water Pump

2.7. Blynk Platform

Blynk is an IoT platform that allows the rapid creation of graphical interfaces for the Remote control and monitoring of embedded devices. Through its mobile application, the Users can view data, receive notifications, and control actuators in real time real through customizable widgets [1]. In this project, Blynk facilitates monitoring of the environmental conditions and the remote management of the irrigation system.

2.8. LCD Screen

The system uses a 16x2 character alphanumeric LCD screen, which is based on the standard HD44780 controller. This screen allows important information to be displayed. such as temperature, humidity, soil moisture, and water consumption in a clear and user-friendly.

Machine Translated by Google



Figure 7: 16x2 LCD Screen

2.9. I2C Communication and PCF8574 Module

To simplify the connection between the ESP32 microcontroller and the LCD screen, the following is used:

An I2C adapter module based on the PCF8574 chip. This module reduces the number of Pins needed to control the screen, using only two data lines, SDA and SCL.

The PCF8574 chip acts as a port expander that allows control of the pins of the LCD via the I2C protocol, facilitating communication and reducing complexity of the wiring in the project.

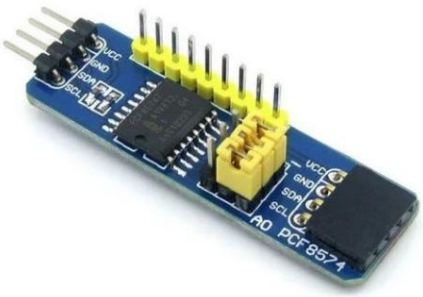


Figure 8: PCF Module

Machine Translated by Google

3. Development

The smart irrigation system was developed using an ESP32 microcontroller and sensors.
environmental sensors, a relay module, and an LCD screen for data display.
The main elements and the implemented operating logic are described below.

3.1. Microcontroller and communication

An ESP32 was used due to its ability to connect to WiFi networks and run the
Blynk platform, which allows remote monitoring and control of the system through a
Mobile application. Communication with sensors and actuators is done via pins
digital and analog, as well as via the I2C bus for the LCD screen.

3.2. Sensors and actuators

The system integrates the following devices:

- Soil moisture sensor connected to analog pin GPIO36, which measures the percentage of moisture present.
- DHT11 temperature and humidity sensor connected to GPIO4, which provides environmental conditions.
- PIR sensor connected to GPIO27, detects movement for additional functions monitoring.
- 5V relay module connected to GPIO26, which controls the power on/off the water pump.
- Physical button on GPIO33 to manually activate or deactivate the relay.

3.3. Program Logic

The code uses the Blynk library to connect the device to the cloud and enable the
Remote interaction. It also uses timers to periodically read the sensors.
and update the LCD screen.

Machine Translated by Google

The water pump is activated or deactivated depending on the state of the physical or virtual button. During
Its operation calculates the approximate water consumption based on the time that
It remains on and has an estimated flow of 1.6 liters per minute.

3.4. Data Visualization

The LCD screen displays real-time temperature, ambient humidity, and humidity
of the soil and accumulated water consumption. The information is updated every second.
to maintain a clear and useful user interface.

3.5. System Operation

The system initiates the WiFi connection and synchronizes with the Blynk platform. Configuration begins.
the pins and sensors, and periodic tasks are set up using timers for
reading and updating data.
In the main loop, the state of the PIR sensor is evaluated to activate indicators of
During movement, Blynk's tasks are executed and user interactions are monitored.
through physical and virtual buttons.
This modular architecture allows the system to be flexible, efficient, and scalable for
future improvements.

Machine Translated by Google

4. Code

This was the code used, as explained above:

Listing 1: PID Speed Code

```
// Viral Science www . viralsciencecreativity. com www. youtube. com /c/ viralscience
// Blynk IOT Smart Plant Monitoring System

# define BLYNK_TEMPLATE_ID " TMPL2I5BNgZpU "
# define BLYNK_TEMPLATE_NAME " Intelligent Irrigation System                    "
# define BLYNK_AUTH_TOKEN " sP_bnZb903gxCO6MNO - h4YMteKdv925c "

# include < Wire .h >
# include < LiquidCrystal_I2C .h >
# include < WiFi .h >
# include < WiFiClient .h >
# include < BlynkSimpleEsp32 .h >
# include < DHT .h >

# define BLYNK_PRINT Serial

// LCD
LiquidCrystal_I2C lcd (0 x20                    ,   16 ,   2) ;

// I2C Pins
const int SDA_PIN = 21;
const int SCL_PIN = 22;

// WiFi and Blynk
char auth [] = BLYNK_AUTH_TOKEN ;
char ssid[] =                 " ********** ";
char pass [] =                " ******** ";

// Blynk Timer
BlynkTimer timer;

// Pins
```

Machine Translated by Google


```
# define SOIL_SENSOR_PIN 36
# define PIR_PIN 27
# define RELAY_PIN_1 26
# define PUSH_BUTTON_1_PIN 33
# define VPIN_BUTTON_1 V12
# define VPIN_CONSUMO V13

// DHT11
# define DHTPIN 4
# define DHTTYPE DHT11
DHT dht ( DHTPIN , DHTTYPE ) ;

// States
int PIR_ToggleValue = 0;
int relay1State = LOW ;
int pushButton1State = HIGH ;

// Consumption
unsigned long onTime = 0; // in seconds
float consumptionLiters = 0.0;
const float LITERS_PER_SECOND = 1.6 / 60.0; // 0.0267

void setup() {
    Wire.begin(SDA_PIN) , SCL_PIN ) ;
    Serial . begin (9600) ;

    lcd . begin (16 , 2) ;
    lcd . backlight () ;
    lcd . setCursor (0 , 0) ;
    lcd. print ("Initializing ");
    for ( int a = 5; a <= 10; a++) {
        lcd.setCursor(a , 1) ;
        lcd . print (".") ;
        delay (500) ;
    }
    lcd . clear () ;
```

Machine Translated by Google


```
    pinMode ( PIR_PIN , INPUT ) ;
    pinMode ( RELAY_PIN_1 , OUTPUT ) ;
    digitalWrite ( RELAY_PIN_1 , relay1State ) ;
    pinMode ( PUSH_BUTTON_1_PIN , INPUT_PULLUP ) ;

    dht . begin () ;
    Blynk.begin(auth , ssid, pass, " blynk . cloud ", 80) ;

    timer . setInterval (100 L , soilMoistureSensor ) ;
    timer . setInterval (2000 L , DHT11sensor ) ;
    timer . setInterval (500 L, checkPhysicalButton);
    timer . setInterval (1000 L , calculateConsumption ) ; // every second
}

// Soil moisture sensor reading
void soilMoistureSensor() {
    float value = analogRead ( SOIL_SENSOR_PIN ) ;
    value = map ( value , 0 , 4095 , 0 , 100) ;
    value = ( value - 100) * -1;
    Blynk . virtualWrite ( V3 , value);
    lcd . setCursor (0 , 1) ;
    lcd . print ("S:") ;
    lcd.print(value);
    lcd . print (" ") ;
    Serial . print (" SOIL MOISTURE: ") ;
    Serial . println ( value ) ;
}

// DHT11 sensor reading
void DHT11sensor () {
    float h = dht . readHumidity () ;
    float t = dht . readTemperature () ;
    if ( isnan ( h ) || isnan ( t ) ) {
        Serial . println (" Error reading DHT11 ") ;
        return ;
    }
```

Machine Translated by Google


```
    Blynk. virtualWrite ( V0 Blynk .        ,   t) ;
    virtualWrite ( V1                       ,   h) ;

    lcd . setCursor (0          ,   0) ;
    lcd . print ("T:") ;
    lcd . print ( t ) ;
    lcd . print ("C ") ;
    lcd . print ("H:") ;
    lcd . print ( h ) ;
    lcd . print (" % ") ;
    Serial. print ("Temp: "); Serial. print ( t ) ;
    Serial. print ("C Hum: "); Serial. print ( h ) ; Serial. println ("%") ;
}

// PIR sensor
void PIRsensor() {
    bool value = digitalRead ( PIR_PIN ) ;
    Serial. print ("PIR Sensor Value: ") ;
    Blynk . virtualWrite ( V5                   ,   value);
    Serial . println ( value ) ;

    if ( value ) {
        Blynk. logEvent ( " pirmotion ", " WARNING ! Motion Detected !") ;
        WidgetLED LED ( V5 ) ;
        LED . on () ;
    } else {
        WidgetLED LED ( V5 ) ;
        LED . off () ;
    }
}

// Physical B
void checkPhysicalButton() {
    if ( digitalRead ( PUSH_BUTTON_1_PIN ) == LOW ) {
        if ( pushButton1State != LOW ) {
            relay1State = ! relay1State ;
            digitalWrite ( RELAY_PIN_1 , relay1State ) ;
```

Machine Translated by Google


```
            Blynk. virtualWrite ( VPIN_BUTTON_1 , relay1State ) ; delay (500) ;


        }
        pushButton1State = LOW ;
    } else {
        pushButton1State = HIGH ;
    }
}

// PIR status from Blynk
BLYNK_WRITE ( V6 ) {
    PIR_ToggleValue = param . asInt () ;
}

// Virtual button connected to relay
BLYNK_CONNECTED () {
    Blynk. syncVirtual ( VPIN_BUTTON_1 ) ;
}

BLYNK_WRITE ( VPIN_BUTTON_1 ) {
    relay1State = param . asInt () ;
    digitalWrite ( RELAY_PIN_1 , relay1State ) ;
}

// Function to calculate water consumption
void calculateConsumption() {
    if ( relay1State == HIGH ) {
        timeOn ++;
        consumptionLiters = ignitionTime * LITERS_PER_SECOND ;
        consumptionLiters = round ( consumptionLiters * 100.0) / 100.0; // round to 2 decimals
    }

    Blynk. virtualWrite (VPIN_CONSUMPTION    ,    consumptionLiters ) ;

    lcd . setCursor (11              ,    1) ;
    lcd . print ("L:") ;
    lcd. print ( consumptionLiters ) ;
```

Machine Translated by Google


```
    lcd.print (" ); // clear leftover text

    Serial.print ("WATER CONSUMPTION: ");
    Serial. print ( consumptionLiters ) ;
    Serial . println (" L") ;
}

// Main loop
void loop () {
    if ( PIR_ToggleValue == 1) {
        lcd . setCursor (5          ,    1) ;
        lcd . print ("M:ON ") ;
        PIRsensor() ;
    } else {
        lcd . setCursor (5          ,    1) ;
        lcd. print ("M:OFF ") ; WidgetLED
        LED (V6);
        LED . off () ;
    }

    Blynk . run () ;
    timer.run();
}
```

Machine Translated by Google

5. Results

During the implementation and testing of the smart irrigation system, it was possible to monitor real-time soil moisture, ambient temperature and humidity, as well as approximate water consumption based on pump operating time.

The consumption calculation was based on an estimated flow of 1.6 liters per minute, obtained from technical specifications of the pump. However, for more accurate measurements, It would have been ideal to use a flow meter that would allow recording the actual flow of water. However, flow meters are often expensive devices to integrate into projects. due to the low budget, the indirect method based on the following was chosen at this stage: the relay's operating time.

The data obtained showed an adequate response of the system to the conditions of the soil and environment, with the ability to manually activate or deactivate the pump The LCD screen provided clear and continuous information to the user, facilitating system status monitoring without the need for a remote connection.

Overall, the results confirm that the system meets the stated objectives. for efficient irrigation monitoring and control, with room for future improvements in accuracy of water consumption.

6. Conclusion

The development of the intelligent irrigation system based on ESP32 proved to be a solution effective for the automated monitoring and control of soil conditions and environment, optimizing water use through precise pump activation.

The use of sensors such as the DHT11 and the soil moisture sensor allowed us to obtain data reliable devices that were successfully integrated with the Blynk platform for monitoring remote control, while local control via a relay and a physical button increased the system versatility.

Although the calculation of water consumption was approximate due to the absence of a For the flow meter, the method based on operating time proved sufficient to estimate the volume used, while maintaining affordable costs.

Machine Translated by Google

The implementation of the LCD screen and the PCF module facilitated direct interaction with the user with the system, improving the experience and on-site control.

Finally, the project opens the door for future improvements, such as the incorporation of More precise sensors and the integration of intelligent algorithms for further optimization irrigation, contributing to a sustainable use of water resources.

Machine Translated by Google

References

[1] Blynk Inc. Blynk documentation. https://docs.blynk.io/en, 2024. Documentation official for integrating IoT sensors and devices with Blynk.

[2] Viral Science - The home of Creativity. New blynk iot smart plant monitoring system. https://www.youtube.com/watch?v=PTJ9sAk2c2I&t=1s&ab_channel=ViralScience-ThehomeofCreativity, 2022. Video guide for implementation Plant monitoring with ESP32 and Blynk.

[3] Espressif Systems. Esp32 series datasheet. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf, 2023. Technical specifications of the ESP32 microcontroller used in the system.

[4] Electrozone Mexico. Mini submersible water pump 3-5v mb-35. https://electrozone.com.mx/tblarticulos_fulltext.php?field=Descripcion&key1=MB-35, 2023. Technical specifications of the pump used in the project.